GPA788 – Projet de session, livrable #1

Tony Wong
Département de génie des systèmes
École de technologie supérieure
1100 Notre-Dame Ouest,
Montréal, Canada
tony.wong@etsmtl.ca

Pierre-Alexandre Monceau
Département de génie des systèmes
École de technologie supérieure
1100 Notre-Dame Ouest,
Montréal, Canada
pierre-alexandre.monceau.1@ens.etsmtl.ca

Noah Boivin
Département de génie des systèmes
École de technologie supérieure
1100 Notre-Dame Ouest,
Montréal, Canada
noah.boivin.1@ens.etsmtl.ca

***Abstract*—Avec l'industrie 4.0, IIoT prend une place centrale. Dans ce contexte, notre projet a pour but de mettre en place un noeud muni de deux capteurs ; un capteur de température et d'humidité relative : le DHT11 et un capteur sonore : le Electret MAX6644. Ces capteurs sont branchés sur un microcontrôleur Arduino Uno. Pour communiquer et obtenir les informations des capteurs, nous allons concevoir des objets en se focalisant sur la sécurité et la fiabilité en respectant notamment les principes d'encapsulation et de gestion des erreurs. Enfin, nous testerons expérimentalement ce noeud afin de valider son bon fonctionnement.**

***Index Terms*—Industrie 4.0, Objets connectés, IIoT, Arduino Uno, DHT11, Electret MAX4466, Conception d'objets**

Code source : https://github.com/PierreAlexandre13/GPA788-ObjetsConnectes

I. Introduction

Avec les récentes avancées des performances des appareils IoT, ceux-ci sont plus petit, plus rapide, plus puissant et plus autonome. Ils représentent maintenant un atout majeur dans la gestion d'un flux de production industriel. Ainsi, l'industrie doit se moderniser et passer à sa version 4.0.

C'est dans ce contexte que prend place notre projet de simulation simplifié d'un pipeline industriel partant des capteurs et allant jusqu'au cloud. L'objectif est ainsi de mettre en place un système d'objets connectés composé de nœuds module-capteur communiquant avec un coordinateur afin de les connectés à un service d'infonuagique et à un ordinateur afin de pouvoir appliquer un algorithme d'apprentissage profond permettant de prédire des séquences de données.

II. Le Hardware (le matériel)

A. Arduino Uno

Nous allons utiliser des Arduinos Uno en tant que microcontrôleurs afin de pouvoir lire des données des capteurs et pour les transmettre au coordinateur. Le microcontrôleur principal est un ATmega328P, possèdant 32 Ko de mémoire Flash, 1 Ko de EEPROM et 2 Ko de RAM. En plus de cela, l'Arduino peut être alimenté et communiquer avec un PC à travers la liaison série USB. Il possède 23 broches E/S, un port I2C, un port SPI ainsi qu'un CAN 10 bits.

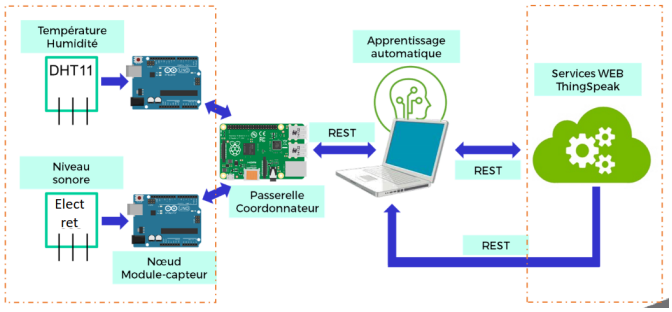


Fig. 1. Système à mettre en place [1]

B. Capteur de température et l'humidité relative

Le capteur utilisé pour mesurer la température et l'humidité relative est un DHT11. Concernant l'humidité relative, sa plage de mesure s'étend de 20% à 90% avec une résolution de 1% et une exactitude de ±5% ; son temps de réponse maximum est de 10 secondes. Concernant la température sa plage de mesure s'étend de 0°C à 50°C avec une résolution de 1°C et une exactitude de ±2°C ; son temps de réponse maximum est de 30 secondes. Il peut être alimenté entre 3.3 et 5 volts. Voici le détail des branchements (attention à bien s'assurer de la présence d'une résistance de tirage, sinon la rajouter) :

Le capteur DHT11 communique ses mesures à travers une liaison série digitale. La trame de données est composée de 80 bits dont la moitié (un sur deux) est un bit de synchronisation. Du reste, il y a 16 bits pour chacune des deux mesures et 8 bits pour la vérification (somme des bits). L'octet le plus important (MSB) représente la partie entière des mesures et l'octet le moins important (LSB) représente la partie fractionnaire. Dans le cas du DHT11, la partie fractionnaire est toujours nulle. Cependant, mieux vaut la prendre tout de même en considération lors des vérifications afin d'éviter des mauvaises surprises (DHT22 vendu en tant que DHT11 par exemple).

C. Capteur de niveau sonore

Le capteur utilisé pour mesurer le niveau sonore est le Microphone Electret MAX4466. Il est caractérisé par un bande passante nominale de 20 Hz à 20 kHz, d'une bande passante effective de 50 Hz à 16 kHz, d'un gain de 25 à 125 ajustable qu'on laissera au maximum. Il peut être alimenté entre 2.4 et

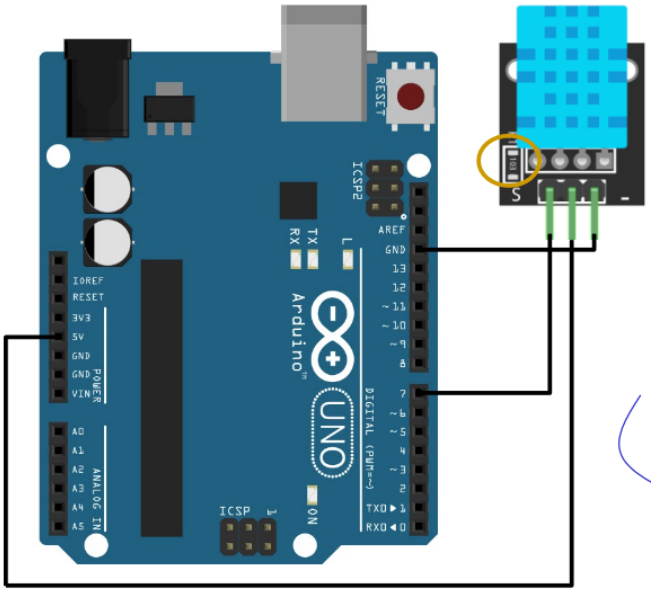


Fig. 2. Branchements capteur DHT11 [1]

5 volts (Vcc) et sa sortie analogique ”rail-to-tail” est comprise entre 0 et Vcc volts, elle comprend un composante continue égale à Vcc/2. Sa sensibilité est autour de -44 dBV/Pa. Voici comment le relier à l'Arduino Uno :

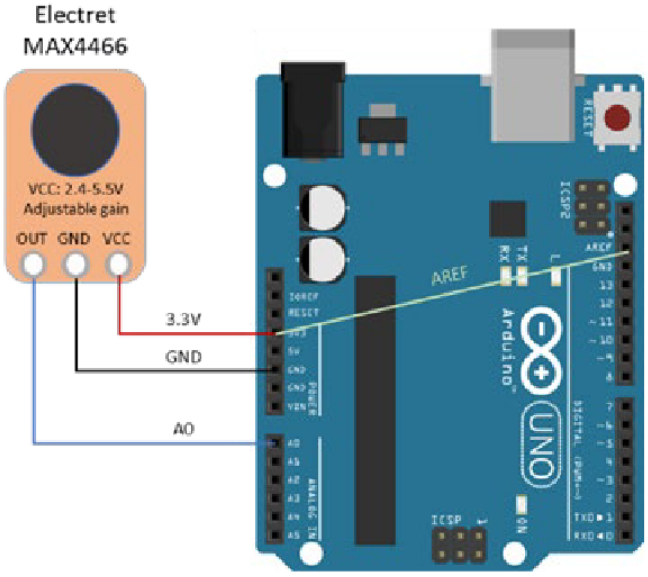


Fig. 3. Branchements capteur Electret MAX4466 [1]

Pour pouvoir obtenir le niveau de bruit équivalent, il va falloir d'abord calculer la valeur RMS du signal puis calculer le niveau sonore pour afin obtenir le niveau de bruit équivalent.

III. Conception des objets

A. Contrôle du capteur DHT11 : Décisions de design

1) Sécurité: Nous avons encapsulé la classe en plaçant certains attributs comme la température et humidité relative et certaines méthodes comme read() et readSensor() dans la section privée de la classe. En conséquence, nous avons aussi dû créer deux accesseurs pour la lecture de ces nouveaux attributs privés étant l'humidité relative et la température.

2) Constructeur et Destructeur: Nous avons décidé d'utiliser le constructeur par défaut que nous avons personnalisé. En effet, nous initialisons les valeurs de la température et de l'humidité relative à des valeurs impossibles comme valeurs initiales afin de ne pas provoquer des lectures erronées. Nous initialisons aussi l'attribut indiquant la connexion du capteur à l'état non connecté et un timer permettant de gérer les fréquences de lecture du capteur.
Nous avons aussi interdit la création par copie, par assignation ou encore par déplacement.
Enfin, nous utilisons le destructeur par défaut vide car il n'y a pas de mémoire à libérer ou d'autres éléments à supprimer.

3) Gestion des erreurs: Sur l'Arduino nous ne pouvons pas utiliser la méthode try/catch/throw pour gérer les erreurs. A la place, nous avons mis en place des codes d'erreurs qui sont retournés par les fonctions en cas d'erreur. Ces codes sont placés dans un ”enum class” nommé `DHTLIB_ErrorCode` afin de réduire les conflits potentiels de nom dans le cas de l'utilisation de plusieurs classes. Parmi les codes, on en retrouve pour indiquer une erreur de checksum, une erreur de timeout, si le capteur est non connecté, si le délai n'est pas respecté et si tout c'est bien passé. Il y a aussi un code de valeur invalide que l'on peut mettre dans les attributs lors d'erreurs.
L'erreur de délai non respecté `DHTLIB_ERROR_DELAI (=-4)` s'active lorsque l'utilisateur n'a pas respecté un délai d'au moins 1 seconde entre deux lectures du capteur.

4) Services supplémentaires: Enfin, nous avons ajouté un système de vérification de la connexion au capteur permettant d'ajouter une couche de protection avec un nouveau attribut booléen et les méthodes suivantes :
- `bool dhtlib_gpa788::isConnected()` : retourne l'état de la connexion avec le capteur
- `void dhtlib_gpa788::Connect()` : met à l'état connecté la variable indiquant la connexion
- `void dhtlib_gpa788::disConnect()` : met à l'état déconnecté la variable indiquant la connexion
- `void dhtlib_gpa788::reset()` : met les attributs de température et d'humidité relative à des valeurs impossibles.

B. Contrôle du capteur Electret MAX4466

1) Organisation et relation des classes: Le capteur Electret MAX4466 est géré par trois classes interconnectés : `calculateur_vrms`, `calculateur_li` et `calculateur_leq`.
Chacune s'occupe de calculer un élément dont la classe suivante a de besoin.

Ainsi, `calculateur_vrms` calcule la tension rms qui représente le niveau sonore à l'entrée du capteur. Elle possède un constructeur personnalisé permettant d'entrer les paramètres du calcul, des accesseurs pour les attributs privés, et deux méthodes publiques : - Accumulate qui permet d'accumuler des échantillons de données capteur et - Compute qui permet d'appliquer la formule de la tension vrms sur ces échantillons.

`calculateur_li`, est basé sur le même principe et utilise le résultat du Compute de calculateur_vrms pour calculer la valeur Li en utilisant la sensibilité et le gain du capteur. Ces paramètres sont aussi renseignés dans son constructeur.

Enfin, `calculateur_leq` utilise les résultats, méthodes et attributs de `calculateur_li` pour calculer le niveau de bruit équivalent.

2) Gestion des temporisations: La partie importante de la classe `calculateur_leq` est la gestion des temporisations car, en effet, elles ne sont pas gérées et sont laissées à la responsabilité de l'utilisateur dans les deux classes précédentes. Cette temporisation représente les périodes d'échantillonnages pour le calcul des trois valeurs. Les périodes d'échantillonnages sont renseignées dans le constructeur. La temporisation est mise en place avec trois timers qui comptent le temps avec la fonction `millis()`.

3) Gestion des erreurs: Enfin, pour la gestion des erreurs du capteur Electret MAX4466 nous avons construit une enum class contenant les codes MAX_OK = 0 lorsqu'il n'y a pas d'erreur et MAX_ERROR = -1 lorsqu'une erreur s'est produite. La détection d'erreur se fait dans la méthode Accumulate() du calculateur_vrms en fonction de la valeur de retour du analogRead(), c'est-à-dire les données provenant directement du capteur. Si la valeur est incohérente, la méthode retourne le code d'erreur -1 et il se propage jusqu'au calculateur_leq qui affiche le code d'erreur à l'utilisateur et reset les timers, ce qui reset le calcul de Leq.

IV. Résultats

A. Capteur DHT11

Afin de vérifier le fonctionnement du capteur DHT11 et de notre code, nous avons ajouté un écran LCD permettant d'afficher les informations importantes comme l'humidité relative, la température ou encore le code d'erreur lorsque nécessaire.

Ainsi, pour vérifier expérimentalement le fonctionnement du capteur, nous avons observé les résultats sur l'écran tout en essayant de chauffer le capteur dans notre main. La température augmente bien, donc le capteur et le code fonctionne.

Pour tester la gestion d'erreur nous pouvons faire exprès d'en provoquer une. Ainsi, retirer le câble de connexion data du capteur provoque bien le code d'erreur -2 et en rebranchant ce câble, l'erreur s'en va et les informations de température et humidité relative s'affichent à nouveau (cf. figure 4).

On peut aussi essayer de mal utiliser la classe en oubliant par exemple d'appeller la méthode Connected() avant de lire les données, ce qui retourne le code d'erreur -3, ou encore en provoquant l'interruption de lecture du capteur toutes les 0.5 secondes (`<1s`) ce qui retourne le code d'erreur -4.

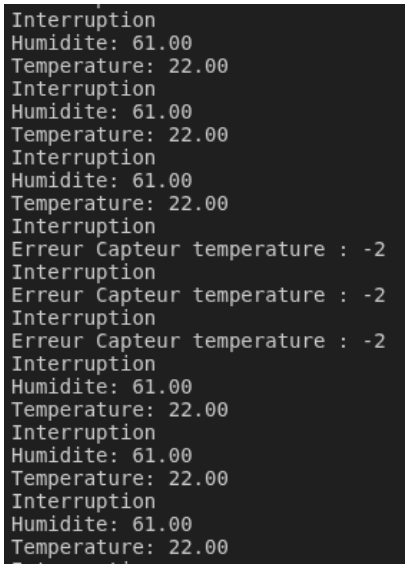


Fig. 4. Résultats de l'utilisation du capteur DHT11 lorsqu'une erreur se produit.

Ainsi, grâce à ses expérimentations, nous pouvons en conclure que notre classe pour manipuler le capteur DHT11 est sécurisée et fiable grâce à la gestion d'erreurs à la fois matérielles et logicielles.

B. Module Electret-MAX4466

Pour vérifier le fonctionnement du module Electret-MAX4466 et de notre classe calculant le niveau de bruit équivalent (Leq), nous avons utilisé le protocole expérimental suivant :

nous avons alimenté le montage puis ouvert le Serial Monitor afin d'observer les mesures. Ensuite, nous pouvons faire du bruit en parlant ou mettant de la musique à moins d'une dizaine de centimètres du capteur et observer alors la valeur de Leq augmenter. Lorsque le bruit s'arrête, la valeur de Leq doit diminuer.

La valeur de Leq se met à jour toutes les cinq minutes mais pour les tests pratique nous pouvons modifier la valeur du paramètre NB_LI à 2 pour obtenir une mise à jour toutes les 2x2=4 secondes.

Voici les paramètres influençant le calcul du niveau de bruit équivalent :

TABLE I
Paramètres utilisés pour le module Electret-MAX4466

Paramètre	Valeur	Signification
P	94 dB SPL	Pression acoustique de référence
M	-44 dBV/Pa	Sensibilité du microphone
G	42 dBV	Gain appliqué au signal
t_s	62 ms	Période d'échantillonnage
N_{sample}	32	Nombre d'échantillons utilisés pour calculer L_i
N_{Li}	$\{2; 10; 150\}$	Nombre de valeurs L_i utilisées pour calculer L_{eq}

Lors de nos expérimentations nous obtenons des résultats cohérents avec des valeurs de niveau de bruit équivalent autour de 44 dB dans une salle silencieuse et nous pouvons constater l'augmentation de cette valeur jusqu'à environ 50 dB lorsque l'on ajoute du bruit.
De plus, nous pouvons tester la gestion d'erreur en débranchant soit le pin data et/ou le pin d'alimentation. Lorsque les valeurs lues ne sont plus cohérentes cela produit le code d'erreur -1.
Nous pouvons bien observer tout cela à la figure 5.
Ainsi, nous en concluons que le module Electret-MAX4466 et notre code fonctionnent correctement.

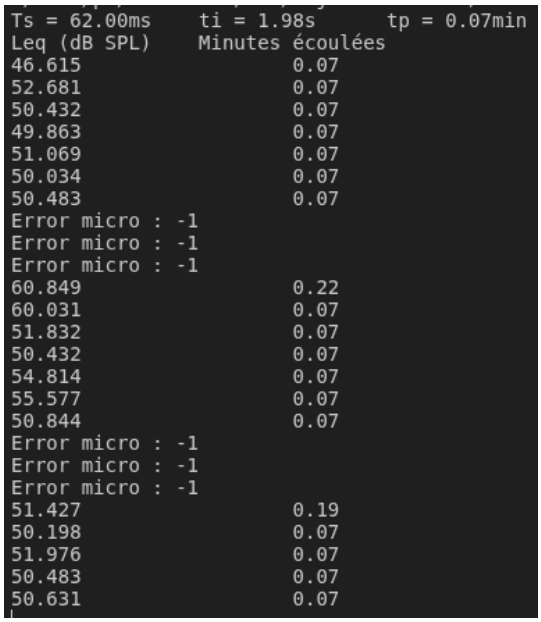


Fig. 5. Résultats de l'utilisation du capteur Electret MAX4466 lorsqu'une erreur se produit et que le niveau sonore augmente.

V. Conclusion

Nous avons vu dans cet article comment mettre en place des noeuds avec des capteurs permettant d'encoder numériquement des grandeurs physiques de leur environnement. Pour cela, nous avons utilisé un microcontrôleur Arduino Uno que nous avons branché avec le capteur de température et d'humidité relative DHT11, ainsi que le capteur sonore Electret MAX4466. Pour pouvoir extraire et calculer les informations dont nous avons besoin, nous avons dû concevoir des objets sous forme de classes en C++. Ces classes ont été construites en respectant les principes d'encapsulation et de gestion des erreurs. Cela permet de rendre notre code plus fiable et sécurisé. Le capteur Electret MAX4466 utilise même trois classes successives afin de calculer le niveau de bruit équivalent. Enfin, grâce à un protocole expérimental nous avons pu tester nos codes et vérifier leur fonctionnement même dans des cas particuliers générateurs d'erreurs.

References

[1] Cours GPA788 de Tony Wong

[2] Documentation officielle d'Arduino